POC 01: Smart Building Management Platform - EKS Stabilization

Prepared for: **Saurabh Dindokar**

Role: **AWS DevOps Engineer**

Public-safe name: **Smart Building Management Platform**

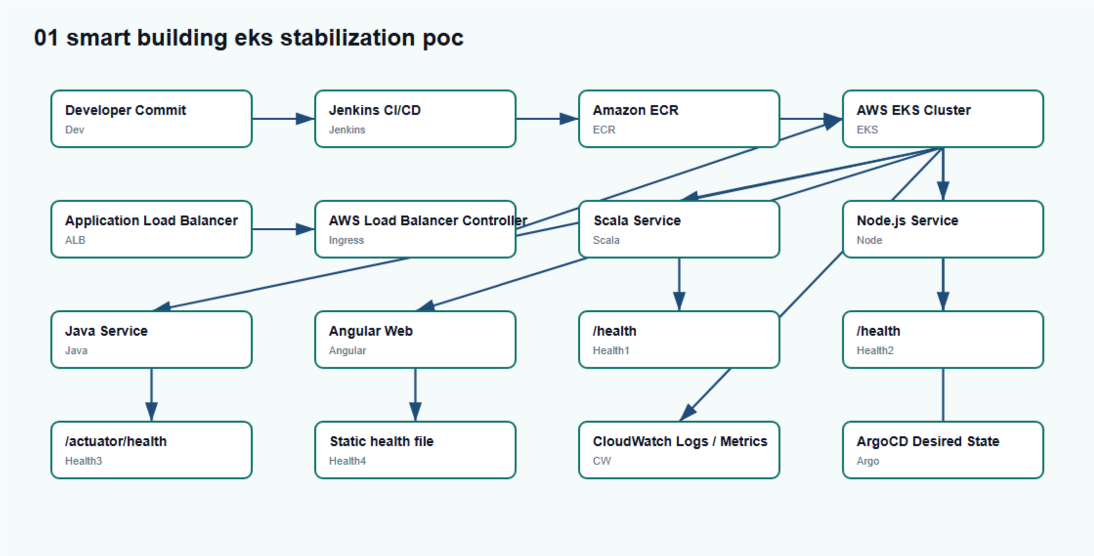
Confidentiality Note

This POC is public-safe. It does not include real client names, internal repository names, production URLs, IP addresses, credentials, private screenshots, or organization-owned source code.

POC Objective

Create a reusable Kubernetes/EKS stabilization blueprint for a multi-service platform with standardized health checks, liveness probes, readiness probes, ingress routing, CloudWatch logging, and deployment handover documentation.

Architecture Diagram



Architecture diagram

What I Worked On

â€¢ Supported AWS EKS migration and Kubernetes workload stabilization planning.

â€¢ Added and documented service health-check endpoints/files across Scala, Node.js, Angular, and Java services.

â€¢ Configured Kubernetes liveness and readiness probe patterns for pod-level stability.

â€¢ Documented ECR image flow, ALB ingress routing, CloudWatch visibility, and operational handover steps.

POC Implementation Scope

â€¢ Dummy services for Scala, Node.js, Java, and Angular health-check behavior.

â€¢ Kubernetes manifests for Deployment, Service, Ingress, ConfigMap, and probe configuration.

â€¢ ALB ingress example using path-based routing.

â€¢ CloudWatch logging and troubleshooting notes.

â€¢ Rollout validation and rollback checklist.

Suggested Repository Structure

```
eks-health-stabilization-poc/
|-- README.md
|-- k8s/
|   |-- namespace.yaml
|   |-- scala-service.yaml
|   |-- node-service.yaml
|   |-- java-service.yaml
|   |-- angular-web.yaml
|   `-- ingress.yaml
|-- docs/
|   |-- runbook.md
|   |-- rollback.md
|   `-- troubleshooting.md
`-- diagrams/
    `-- architecture.mmd
```

Validation Steps

Deploy dummy workloads to a local Kubernetes cluster or test EKS namespace.

Confirm all pods become ready only after readiness checks pass.

Simulate a failed health endpoint and confirm liveness restart behavior.

Confirm ingress routes traffic only to ready pods.

Review logs and events for troubleshooting documentation.

Expected Outcome

â€¢ Safer Kubernetes rollouts.

â€¢ Better pod restart and readiness behavior.

â€¢ Clear health-check standard for future services.

â€¢ Better handover documentation for support teams.

Technologies

AWS EKS, Kubernetes, Docker, Amazon ECR, ALB Ingress, AWS Load Balancer Controller, Jenkins, ArgoCD, CloudWatch, Scala, Node.js, Angular, Java.